

Station Météo et Surveillance de Réservoir — ESP32

Guide de développement complet

Windows 10/11 • Python • Arduino/PlatformIO • Railway

0. Table des matières

1	Contexte et objectif du projet	1
2	Architecture	2
3	Logiciels à installer	3
4	Étape 1 — Valider l’environnement ESP32	3
5	Étape 2 — Connexion Wi-Fi de l’ESP32	3
6	Étape 3 — Câblage des capteurs et de la LED	4
6.1	DHT11 — Température et humidité	4
6.2	HC-SR04 — Niveau d’eau du réservoir	4
6.3	LED d’alerte	4
7	Étape 4 — Backend FastAPI	4
7.1	Installation	5
7.2	Base de données — database.py	5
7.3	API — main.py	5
8	Étape 5 — Firmware complet ESP32	5
9	Étape 6 — Dashboard Web	6
10	Étape 7 — Déploiement sur Railway	6
10.1	Préparer les fichiers	6
10.2	Pousser sur GitHub	6
10.3	Déployer sur Railway	7
11	Dépannage et points d’attention	7
12	Checklist de validation finale	7

1. Contexte et objectif du projet

Le projet consiste à connecter une carte ESP32 à Internet afin de surveiller l’environnement d’un espace et le niveau d’un réservoir d’eau, et d’exposer ces données sur un dashboard web accessible depuis n’importe où.

Problème résolu : comment surveiller à distance la température, l’humidité et le niveau d’eau d’un réservoir, recevoir une alerte visuelle locale en cas d’anomalie, et piloter un équipement (LED, ventilateur) depuis un téléphone ?

Ce que fait le système :

- L'ESP32 mesure la température et l'humidité (capteur DHT11) toutes les 10 secondes et envoie les données à un serveur via HTTP.
- L'ESP32 mesure le niveau d'eau du réservoir (capteur ultrason HC-SR04) et allume une LED d'alerte si le niveau descend sous un seuil.
- Le serveur stocke les données et met à disposition des commandes (allumer/éteindre la LED depuis le dashboard).
- Un dashboard web affiche l'historique des mesures, l'état de connexion de la carte, et permet d'envoyer des commandes à distance.

Cas d'usage concrets :

- Serre agricole : alerte si température trop haute ou réservoir vide.
- Chambre ou bureau : suivi du confort thermique et hygrométrique.
- Château d'eau ou cuve : surveillance du niveau en temps réel.

Matériel utilisé : ESP32 DevKit V1, capteur DHT11 (température + humidité), capteur ultrason HC-SR04 (niveau d'eau), LED + résistance 330 Ω , breadboard, fils.

Système d'exploitation : Windows 10/11.

2. Architecture

Le projet suit une architecture 4 couches classique :

ESP32 $\longrightarrow$ FastAPI (Python) $\longrightarrow$ SQLite $\longrightarrow$ Dashboard HTML/JS

Principe clé : le frontend HTML est servi directement par FastAPI. Il n'y a donc *qu'un seul serveur* à déployer. L'ESP32 et le navigateur pointent vers la même URL publique.

Structure des fichiers :

```
mon-projet-iot/
  backend/
    main.py          (application FastAPI)
    database.py      (modèles SQLite)
    requirements.txt
    Procfile         (pour Railway)
    static/
      index.html     (dashboard web)
  esp32/
    main.cpp         (firmware ESP32)
    platformio.ini
    include/
      credentials.h  (SSID/PASSWORD - ignoré par .gitignore)
  .gitignore
  README.md
```

Flux de données :

1. ESP32 POST /data $\rightarrow$ FastAPI enregistre dans SQLite (température, humidité, niveau d'eau).
2. Dashboard GET /history toutes les 5s $\rightarrow$ graphiques mis à jour.

3. Dashboard `POST /command` → commande stockée en BDD.
4. ESP32 `GET /command` toutes les 5s → exécute si commande en attente (`LED_ON / LED_OFF`).
5. ESP32 : si niveau d'eau < seuil défini, allume la LED d'alerte localement sans attendre de commande.
6. Dashboard `GET /status` → online si dernière donnée < 30s.

3. Logiciels à installer

- **VS Code** — <https://code.visualstudio.com> (cocher *Add to PATH*)
- **Extension PlatformIO** dans VS Code (`Ctrl+Shift+X` → *PlatformIO IDE*)
- **Driver USB ESP32** — CP2102 : <https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers> ou CH340 : <https://sparks.gogo.co.nz/ch340.html>
- **Python 3.11+** — <https://www.python.org/downloads> (cocher *Add Python to PATH*)
- **Git** — <https://git-scm.com/download/win>

Comptes en ligne nécessaires : **GitHub** (<https://github.com>) et **Railway** (<https://railway.app>).

4. Étape 1 — Valider l'environnement ESP32

Objectif : faire clignoter la LED intégrée pour confirmer que la chaîne compile et flashe correctement. *Durée estimée : 30 min.*

1. Dans VS Code → icône PlatformIO → **New Project**.
2. Board : *Espressif ESP32 Dev Module* — Framework : *Arduino*.
3. Dans `src/main.cpp`, écrire un programme qui alterne `digitalWrite(2, HIGH/LOW)` avec `delay(1000)` et affiche un message via `Serial.println`.
4. Cliquer sur la flèche Upload. Si erreur de port, ajouter `upload_port = COM3` dans `platformio.ini`.
5. Ouvrir le Moniteur Série → les messages doivent apparaître.

Validation : LED clignote + messages dans le moniteur série.

5. Étape 2 — Connexion Wi-Fi de l'ESP32

Objectif : connecter l'ESP32 au réseau et afficher son adresse IP. *Durée estimée : 20 min.*

Créer le fichier `include/credentials.h` (listé dans `.gitignore`) :

```
#define WIFI_SSID "VotreSSID"
#define WIFI_PASS "VotreMotDePasse"
```

Dans `platformio.ini`, déclarer les librairies nécessaires :

```
lib_deps =
    adafruit/DHT sensor library@~1.4.4
    adafruit/Adafruit Unified Sensor@~1.1.9
    bblanchon/ArduinoJson@~7.0.0
```

Le firmware doit :

- Appeler `WiFi.begin(WIFI_SSID, WIFI_PASS)` dans `setup()`.
- Attendre `WiFi.status() == WL_CONNECTED` en boucle (max 20 tentatives).
- Afficher `WiFi.localIP()` dans le moniteur série.
- Tenter une reconnexion automatique dans `loop()` si la connexion est perdue.

Validation : l'adresse IP s'affiche dans le moniteur série.

6. Étape 3 — Câblage des capteurs et de la LED

Objectif : brancher le DHT11, le HC-SR04 et la LED sur l'ESP32. *Durée estimée : 20 min.*

6.1 DHT11 — Température et humidité

- VCC → 3.3 V de l'ESP32.
- GND → GND.
- DATA → GPIO 4 (avec résistance pull-up 10 kΩ entre DATA et 3.3 V).

6.2 HC-SR04 — Niveau d'eau du réservoir

Le HC-SR04 fonctionne en 5 V. La broche Echo renvoie un signal 5 V qui doit être abaissé à 3.3 V avant d'entrer sur l'ESP32, sous peine d'endommager la carte.

- VCC → Vin (5 V).
- GND → GND.
- Trig → GPIO 5 (pas de pont diviseur nécessaire en sortie).
- Echo → pont diviseur résistif ($R_1 = 1\text{ k}\Omega$, $R_2 = 2\text{ k}\Omega$) → GPIO 18.

Principe de mesure du niveau : le capteur est placé au-dessus du réservoir, face vers le bas. La distance mesurée correspond à l'espace entre le capteur et la surface de l'eau. Plus la distance est grande, plus le niveau est bas.

$$\text{niveau (\%)} = 100 \times (1 - \text{distance_mesurée} / \text{hauteur_réservoir})$$

6.3 LED d'alerte

- Anode → GPIO 2 via résistance 330 Ω.
- Cathode → GND.
- La LED s'allume automatiquement si le niveau d'eau descend sous 20 % ou si une commande `LED_ON` est reçue du dashboard.

Validation : lire la distance dans le moniteur série, vérifier que la valeur est cohérente avec la position du capteur.

7. Étape 4 — Backend FastAPI

Objectif : serveur Python avec API REST et base SQLite. *Durée estimée : 45 min.*

7.1 Installation

```
mkdir backend && cd backend
python -m venv venv
venv\Scripts\activate
pip install fastapi uvicorn sqlalchemy python-multipart
```

7.2 Base de données — database.py

Deux tables SQLite via SQLAlchemy :

- **sensor_data** : colonnes `id`, `device_id`, `temperature` (float), `humidity` (float), `water_level` (float, en %), `timestamp`.
- **commands** : colonnes `id`, `device_id`, `command` (string), `executed` (0 ou 1), `created_at`.

7.3 API — main.py

Cinq routes à implémenter :

- POST `/data` — reçoit `{device_id, temperature, humidity, water_level}`, insère dans `sensor_data`.
- GET `/history?limit=50` — retourne les N dernières mesures triées par timestamp.
- POST `/command` — reçoit `{device_id, command}`, insère avec `executed=0`.
- GET `/command?device_id=esp32-1` — retourne la prochaine commande non exécutée et la marque `executed=1`.
- GET `/status` — retourne `online: true/false` selon que la dernière donnée date de moins de 30 secondes.

FastAPI sert aussi le fichier `static/index.html` via `StaticFiles` et une route GET `/` pointant vers ce fichier.

Lancer en local :

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Tester les routes sur `http://localhost:8000/docs`.

Validation : POST `/data` retourne `{"status":"ok"}`, GET `/history` retourne la liste des mesures.

8. Étape 5 — Firmware complet ESP32

Objectif : l'ESP32 mesure, allume la LED si besoin, envoie les données et reçoit des commandes.

Durée estimée : 45 min.

Le `loop()` tourne en permanence avec deux intervalles :

- Toutes les **10 secondes** :
 1. Lire DHT11 : `float t = dht.readTemperature();` et `float h = dht.readHumidity();`.
 2. Mesurer HC-SR04 : impulsion Trig 10 μ s, lire durée sur Echo, calculer `distance = duration * 0.034 / 2`.
 3. Calculer le niveau : `level = 100 * (1 - distance / RESERVOIR_HEIGHT)`.
 4. Allumer la LED si `level < 20` (alerte locale, sans attendre de commande).

- 5. S rialiser en JSON et faire un POST `/data`.
- Toutes les **5 secondes** : faire un GET `/command`, parser le JSON retourn , ex cuter LED_ON ou LED_OFF si la commande n'est pas NONE.

En production, pointer `SERVER_URL` vers l'URL Railway. En local, pointer vers l'IP du PC (`ipconfig` dans cmd).

Validation : messages [OK] Donn es envoy es dans le moniteur s rie, LED s'allume quand le capteur est tenu loin d'une surface, et la LED r pond aux commandes envoy es via Swagger.

9.  tape 6 — Dashboard Web

Objectif : page HTML unique dans `backend/static/index.html`. *Dur e estim e : 1 h.*

La page doit contenir quatre sections :

- **Statut** : badge *En ligne / Hors ligne* aliment  par GET `/status`.
- **Valeurs actuelles** : temp rature, humidit  et niveau d'eau affich s en grand, mis   jour toutes les 5 secondes.
- **Graphiques** : courbes de l'historique avec Chart.js (<https://cdn.jsdelivr.net/npm/chart.js>), rafra chies toutes les 5 secondes via GET `/history`.
- **Contr le LED** : deux boutons qui appellent POST `/command` avec `command`: "LED_ON" ou "LED_OFF".

Tout est en JavaScript vanilla (`fetch API`), sans framework. Le rafra chissement automatique se fait avec `setInterval(fn, 5000)`.

Validation : ouvrir `http://localhost:8000` et voir les graphiques se mettre   jour en temps r el.

10.  tape 7 — D ploiement sur Railway

Objectif : rendre l'application accessible publiquement. *Dur e estim e : 30 min.*

10.1 Pr parer les fichiers

Ajouter dans `backend/` le fichier `Procfile` (sans extension) :

```
web: uvicorn main:app --host 0.0.0.0 --port $PORT
```

Cr er `.gitignore`   la racine :

```
*.db
__pycache__/
venv/
credentials.h
```

10.2 Pousser sur GitHub

```
git init
git add .
git commit -m "Initial commit"
```

```
git remote add origin https://github.com/USERNAME/esp32-meteo.git
git push -u origin main
```

10.3 Déployer sur Railway

1. Se connecter sur <https://railway.app> avec GitHub.
2. **New Project** → **Deploy from GitHub repo** → sélectionner le dépôt.
3. Dans les Settings du service, définir **Root Directory** : **backend**.
4. Cliquer **Deploy** (2–3 minutes).
5. **Settings** → **Networking** → **Generate Domain** → copier l'URL publique.

Mettre à jour `SERVER_URL` dans `credentials.h` avec l'URL Railway, recompiler et re-flasher l'ESP32.

Note : SQLite est réinitialisé à chaque redéploiement sur Railway (système de fichiers éphémère). Acceptable pour un projet de démonstration. Pour persister les données, utiliser le plugin PostgreSQL de Railway.

Validation : ouvrir l'URL publique dans un navigateur, voir le dashboard avec les données de l'ESP32 en direct.

11. Dépannage et points d'attention

- **ESP32 non détecté** : vérifier le driver CP2102/CH340 et le câble USB (certains câbles sont alimentation seule, sans données).
- **Erreur de port à l'upload** : ajouter `upload_port = COMx` dans `platformio.ini`.
- **DHT11 retourne NaN** : vérifier la résistance pull-up 10 k Ω entre DATA et 3.3 V.
- **Distance HC-SR04 toujours -1** : vérifier que le HC-SR04 est alimenté en 5 V (pas 3.3 V) et que le pont diviseur est bien câblé sur Echo.
- **Niveau d'eau incohérent** : définir `RESERVOIR_HEIGHT` en cm selon la hauteur réelle du réservoir.
- **Erreur CORS dans le dashboard** : ajouter `CORSMiddleware` dans FastAPI avec `allow_origins=["*"]`.
- **Railway build failed** : vérifier que **Root Directory** est bien **backend** et que `requirements.txt` est présent.
- **Ne jamais commiter les identifiants Wi-Fi** : ils doivent rester dans `credentials.h` listé dans `.gitignore`.

12. Checklist de validation finale

LED intégrée clignote (étape 1 validée)
Adresse IP s'affiche dans le moniteur série
DHT11 retourne une température et une humidité cohérentes
HC-SR04 retourne une distance valide dans le moniteur série
LED s'allume automatiquement quand le niveau d'eau est bas
POST /data et GET /history fonctionnent via Swagger
ESP32 envoie des données toutes les 10 secondes
Commandes LED_ON/LED_OFF reçues et exécutées
Dashboard affiche les graphiques en temps réel
Badge En ligne / Hors ligne correct
URL Railway publique et accessible

README avec lien de démo sur GitHub
Identifiants Wi-Fi absents du dépôt